

Homework1

a. List vs Set

List and Set are interface used to store groups of objects.

List maintains insertion order, allows duplicate elements, support index-based access and is auto-increasing capacity.

Set doesn't allow duplicate elements.

b. LinkedList vs ArrayList

LinkedList is implemented using doubly linked node, so insertion order so insertion and deletion in the middle of a list is efficient cause elements do not need to be shifted.

ArrayList is implemented using a dynamic array, so it provide fast random access using indexes.

c. What is Map Interface

Map Interface includes LinkedHashMap, TreeMap, ConcurrentMap.

LinkedHashMap: keeps the insertion order of the key

TreeMap: sorting based on the keys

ConcurrentMap keep thread safe.

d. How does HashMap work

HashMap stores data as a key value pairs. When inserting a key, HashMap first calculate the key's hashCode and converts it into a bucket. If bucket is empty, it stored directly.

If the entry is already exists in same bucket, HashMap uses equals() to compare keys. If keys are equal it replace the old value. If the key is different, it handles the collision by sorting multiple entries in the same bucket using a linked list or red-black tree.

e. What is hash collision

Hash collision happens when two objects have the same hash values and are stored in the same bucket. HashMap handles collisions using linked lists or red-black tree which improves worst-case performance from $O(n)$ to $O(\log n)$.

f. What is Collections used for

Collections is used to store and manage objects as a group such as List, Set, and Queue for sorting and searching.

List contains LinkedList, and ArrayList.

List allows duplicate and store elements in insertion order.

Set contains HashSet, LinkedHashSet, TreeSet.

Set doesn't allow duplicate elements.

HashSet stores unordered unique elements.

TreeSet stores customized sorting order elements.

Stack follows last in first out.

Queue contains Queue, Deque, ArrayDeque.

Queue follows first in first out.

Deque is double ended queue allows insertion and deletion from both side.

- g. What is immutable class

Immutable class is a class whose objects cannot be modified after creation.

It improves thread safety, security, and consistency because the object state remains constant.

Make class final, make fields private and final, no setter methods, initialize values through constructor and return copies of mutable objects.

- h. Hashtable vs HashMap vs ConcurrentHashMap

Hashtable is thread safe cause it entire table is synchronized, but it has low performance.

HashMap calculate hashCode() of key and convert hash to bucket index store as key-value pair and is not thread safe.

ConcurrentHashMap is thread safe with bucket level synchronization instead of lock entire table, so it's recommend for multithreading.

- i. String vs StringBuilder vs StringBuffer

String is immutable and will store at constant String pool once created so use it when object doesn't need many modification.

StringBuilder is mutable and not thread safe so use it when many modifications at one thread.

StringBuffer is mutable and thread safe so use it when multiple thread modify the same string objects.

- j. why we need to override the hashCode and equals method at the same time
We override both method at the same time cause hash-based collections like HashMap and HashSet use hashCode() to find the bucket and equals() to compare objects inside the bucket.

If only **equal()** is overridden, two logical equal objects may have different hashCode and store as different bucket cause duplicate keys or lookup failures..
If only **hashCode** is overridden, objects with the same hash code may still not be considered equal correctly.

- k. play around the common data structure apis (map, set, queue, list), write some practice codes.

```
List<Integer> list = new ArrayList<>();  
list.add("Apple");  
System.out.print(list.get(0));  
list.set(0,"kiwi");
```

```
Set<Integer> set = new HashSet<>();  
set.add(20);  
set.add(10);  
System.out.print(set.contains(20));  
set.add(10);
```

```
Map<String, Integer> map = new HashMap<>();  
map.put("Red",1);  
map.put("Blue",2);  
System.out.print(map.get("Blue"));
```

```
Queue<String> queue = new LinkedList<>();  
queue.offer("A");  
queue.offer("B");  
System.out.print(queue.size());
```

Homework2

- a. String vs StringBuilder vs StringBuffer
String is immutable and will store at constant String pool once created so use it when object doesn't need many modification.
StringBuilder is mutable and not thread safe so use it when many modifications at one thread.
StringBuffer is mutable and thread safe so use it when multiple thread modify the same string objects.
- b. Comparator vs Comparable, when to use which one
Both of them are interface used for sorting objects.
Comparator defines customer ordering outside the class using compare() method with multiple sorting logic.
Comparable defines natural ordering of objects inside the class using compareTo() method with one default sorting logic.
- c. Overriding vs overloading
Overriding: runtime polymorphism, subclass provides its own implementation of parent method.
Overloading: compile time polymorphism, occurs when same method with different signature(parameters)
- d. JRE vs JDK vs JVM
- e. Java 8 basic data types
It includes Primitive types and reference types.
Primitive type(store simple values efficiently and faster than reference data type)
Integer Types: byte, short, int, long.
Floating point type: float, double.
Character type: char.
Boolean type: boolean.

Reference type store object references.
String is a class reference type not primitive type.
Wrapper class includes Integer, Character, Double, Boolean. It used for Collections, Generics, and Utility methods.
- f. Primitive type, reference type
Primitive type specifies the type of a variable includes byte, short, int, long, float, double, boolean, char.
Reference type is all the other data types than Primitive type.

- g. How does JVM work
- h. JVM memory data model
- i. How does GC work
- j. young/old/perm generation
- k. difference types of GC